

ASTRE - Lab Assignment 2

Amy van der Meijden (5301513) and Jimmy Oei (6540031)

May 2026

Task 1.

We re-ran Problems 11, 12, 13, 14, 15 and 17 with our concolic execution implementation and compared it against the AFL results from the previous lab. Figure 1 shows the convergence curves of both strategies, and Figure 2 the final number of unique error codes found.

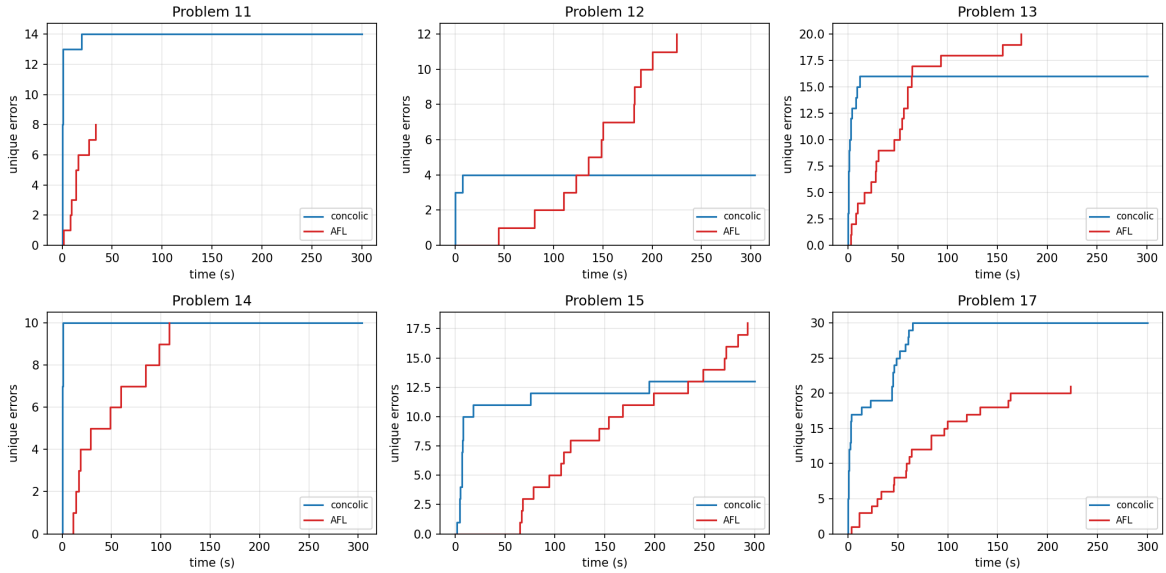


Figure 1: Unique error codes vs. time (5 min budget) for the concolic fuzzer (blue) and AFL (red).

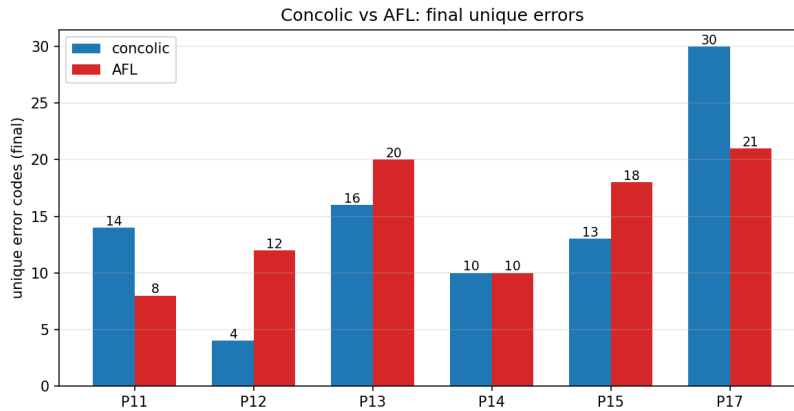


Figure 2: Final unique error counts after 5 min.

We see that concolic does better for Problem 11 and Problem 17, and ties with AFL on problem 14. However, for the other problems AFL finds more unique errors. Noticable is the manner in which the different algorithms converge. Concolic finds a burst of unique errors at the start and then converges at a certain point, resulting in a sharp vertical line that becomes plateaus at a certain point. The AFL however starts off slower, but keeps climbing steadily. It doesn't look like it is plateauing, so likely given more time it would keep climbing and find more unique errors. AFL seems the better choice if your goal is to find as many unique errors as possible and you have to time to run it. Concolic seems best if you quickly want to test a decent amount of unique errors, and don't necessarily need to find the maximum amount.

Task 2: KLEE vs AFL

1. AFL is a mutation-based fuzzer that mutates inputs and uses coverage feedback to guide exploration. KLEE is a symbolic execution engine that builds path constraints and uses an SMT solver to find inputs that reach new branches. AFL finds shallow bugs quickly and scales well; KLEE finds bugs requiring precise inputs deterministically, but suffers from path explosion.
2. Table 1 shows the results for both tools with a 5-minute budget.

Problem	KLEE (unique)	AFL (unique)
11	18	8
12	13	12
13	2	20
14	3	10
15	0	18
17	10	21

Table 1: Unique errors found by KLEE and AFL within a 5-minute budget.

3. AFL outperforms KLEE on most problems, especially 13 (2 vs 20), 15 (0 vs 18), and 17 (10 vs 21). KLEE only wins on Problem 11 (18 vs 8). The poor KLEE results on larger problems are explained by path explosion – Problems 14, 15 and 17 have alphabets of 15–20 symbols, causing exponential state growth. During experiments KLEE repeatedly hit its memory cap and killed states, limiting coverage. Problem 15 found 0 errors entirely for this reason.

KLEE's advantage on Problem 11 shows its strength: it finds errors requiring precise inputs deterministically and fast (e.g. `error_60` in 1.3s), while AFL relies on random mutation. AFL's consistent performance across all problems makes it the better general-purpose tool, while KLEE is most effective on smaller, well-contained problems.

References

- [1] Source code repository: <https://github.com/AmyAmyy/ASTRE26/tree/Lab2>